

ME314 Homework 3

Submission instructions

Deliverables that should be included with your submission are shown in **bold** at the end of each problem statement and the corresponding supplemental material. **Your homework will be graded IFF you submit a single PDF, .mp4 videos of animations when requested and a link to a Google colab file that meet all the requirements outlined below.**

- List the names of students you've collaborated with on this homework assignment.
- Include all of your code (and handwritten solutions when applicable) used to complete the problems.
- Highlight your answers (i.e. **bold** and outline the answers) for handwritten or markdown questions and include simplified code outputs (e.g. `.simplify()`) for python questions.
- Enable Google Colab permission for viewing
- Click Share in the upper right corner
- Under "Get Link" click "Share with..." or "Change"
- Then make sure it says "Anyone with Link" and "Editor" under the dropdown menu
- Make sure all cells are run before submitting (i.e. check the permission by running your code in a private mode)
- Please don't make changes to your file after submitting, so we can grade it!
- Submit a link to your Google Colab file that has been run (before the submission deadline) and don't edit it afterwards!

NOTE: This Juputer Notebook file serves as a template for you to start homework. Make sure you first copy this template to your own Google driver (click "File" -> "Save a copy in Drive"), and then start to edit it.

```
In [4]: #IMPORT CELL
import sympy as sym
import numpy as np
import matplotlib.pyplot as plt
```

```
In [5]: #####
# If you're using Google Colab, uncomment this section by selecting the whole section and press
# ctrl+'/' on your and keyboard. Run it before you start programming, this will enable the nice
# LaTeX "display()" function for you. If you're using the local Jupyter environment, leave it alone
#####

def custom_latex_printer(exp,**options):
    from google.colab.output._publish import javascript
    url = "https://cdnjs.cloudflare.com/ajax/libs/mathjax/3.1.1/latest.js?config=TeX-AMS_HTML"
    javascript(url=url)
    return sym.printing.latex(exp,**options)
sym.init_printing(use_latex="mathjax",latex_printer=custom_latex_printer)
```

Below are the help functions in previous homeworks, which you may need for this homework.

```
In [6]: def integrate(f, xt, dt):
        """
        This function takes in an initial condition x(t) and a timestep dt,
        as well as a dynamical system f(x) that outputs a vector of the
        same dimension as x(t). It outputs a vector x(t+dt) at the future
        time step.
```

```

Parameters
=====
dyn: Python function
    derivate of the system at a given step x(t),
    it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
xt: NumPy array
    current step x(t)
dt:
    step size for integration

Return
=====
new_xt:
    value of x(t+dt) integrated from x(t)
"""
k1 = dt * f(xt)
k2 = dt * f(xt+k1/2.)
k3 = dt * f(xt+k2/2.)
k4 = dt * f(xt+k3)
new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
return new_xt

def simulate(f, x0, tspan, dt, integrate):
    """
    This function takes in an initial condition x0, a timestep dt,
    a time span tspan consisting of a list [min_time, max_time],
    as well as a dynamical system f(x) that outputs a vector of the
    same dimension as x0. It outputs a full trajectory simulated
    over the time span of dimensions (xvec_size, time_vec_size).

    Parameters
    =====
    f: Python function
        derivate of the system at a given step x(t),
        it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
    x0: NumPy array
        initial conditions
    tspan: Python list
        tspan = [min_time, max_time], it defines the start and end
        time of simulation
    dt:
        time step for numerical integration
    integrate: Python function
        numerical integration method used in this simulation

    Return
    =====
    x_traj:
        simulated trajectory of x(t) from t=0 to tf
    """
    N = int((max(tspan)-min(tspan))/dt)
    x = np.copy(x0)
    tvec = np.linspace(min(tspan),max(tspan),N)
    xtraj = np.zeros((len(x0),N))
    for i in range(N):
        xtraj[:,i]=integrate(f,x,dt)
        x = np.copy(xtraj[:,i])
    return xtraj

def animate_double_pend(theta_array,L1=1,L2=1,T=10):

```

```

"""
Function to generate web-based animation of double-pendulum system

Parameters:
=====
theta_array:
    trajectory of theta1 and theta2, should be a NumPy array with
    shape of (2,N)
L1:
    length of the first pendulum
L2:
    length of the second pendulum
T:
    length/seconds of animation duration

Returns: None
"""

#####
# Imports required for animation.
from plotly.offline import init_notebook_mode, iplot
from IPython.display import display, HTML
import plotly.graph_objects as go

#####
# Browser configuration.
def configure_plotly_browser_state():
    import IPython
    display(IPython.core.display.HTML('''
        <script src="/static/components/requirejs/require.js"></script>
        <script>
            requirejs.config({
                paths: {
                    base: '/static/base',
                    plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                },
            });
        </script>
        '''))
configure_plotly_browser_state()
init_notebook_mode(connected=False)

#####
# Getting data from pendulum angle trajectories.
xx1=L1*np.sin(theta_array[0])
yy1=-L1*np.cos(theta_array[0])
xx2=xx1+L2*np.sin(theta_array[0]+theta_array[1])
yy2=yy1-L2*np.cos(theta_array[0]+theta_array[1])
N = len(theta_array[0]) # Need this for specifying length of simulation

#####
# Using these to specify axis limits.
xm=np.min(xx1)-0.5
xM=np.max(xx1)+0.5
ym=np.min(yy1)-2.5
yM=np.max(yy1)+1.5

#####
# Defining data dictionary.
# Trajectories are here.

```

```

data=[dict(x=xx1, y=yy1,
           mode='lines', name='Arm',
           line=dict(width=2, color='blue')
           ),
       dict(x=xx1, y=yy1,
           mode='lines', name='Mass 1',
           line=dict(width=2, color='purple')
           ),
       dict(x=xx2, y=yy2,
           mode='lines', name='Mass 2',
           line=dict(width=2, color='green')
           ),
       dict(x=xx1, y=yy1,
           mode='markers', name='Pendulum 1 Traj',
           marker=dict(color="purple", size=2)
           ),
       dict(x=xx2, y=yy2,
           mode='markers', name='Pendulum 2 Traj',
           marker=dict(color="green", size=2)
           ),
       ]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
            yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor = "x",dtick=1),
            title='Double Pendulum Simulation',
            hovermode='closest',
            updatemenus= [{ 'type': 'buttons',
                            'buttons': [{ 'label': 'Play','method': 'animate',
                                           'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
                                           {'args': [[None], {'frame': {'duration': T, 'redraw': False}, 'mode': 'immediate',
                                           'transition': {'duration': 0}}], 'label': 'Pause','method': 'animate'}
                            ]
                        })

#####
# Defining the frames of the simulation.
# This is what draws the lines from
# joint to joint of the pendulum.
frames=[dict(data=[dict(x=[0,xx1[k],xx2[k]],
                        y=[0,yy1[k],yy2[k]],
                        mode='lines',
                        line=dict(color='red', width=3)
                        ),
                    go.Scatter(
                        x=[xx1[k]],
                        y=[yy1[k]],
                        mode="markers",
                        marker=dict(color="blue", size=12)),
                    go.Scatter(
                        x=[xx2[k]],
                        y=[yy2[k]],
                        mode="markers",
                        marker=dict(color="blue", size=12)),
                    ]) for k in range(N)]

#####

```

```
# Putting it all together and plotting.  
figure1=dict(data=data, layout=layout, frames=frames)  
iplot(figure1)
```

Problem 1 (10pts)

Let $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ with $f(x, y) = \sin(x + y) \sin(x - y)$. Show that $(x, y) = (0, \pi/2)$ satisfies both the necessary and sufficient conditions to be a local minimizer of f .

Hint 1: You will need to take the first and second order derivative of f with respect to $[x, y]$.

Turn in: A scanned (or photograph from your phone or webcam) copy of your hand written solution. You can also use $\text{\textit{LATEX}}$. If you use SymPy, include a copy of your code and all the outputs. Regardless of the format you choose, explain why your result satisfies the necessary and sufficient conditions.

$$1) \quad f(x, y) = \sin(x+y) \sin(x-y)$$

$$f(x, y) = \frac{\cos(2y) - \cos(2x)}{2}$$

$$Df(x, y) = \left(\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \right)^T = (\sin 2x \quad -\sin 2y)^T$$

$$D^2 f(x, y) = \begin{bmatrix} \frac{\partial}{\partial x} (\sin 2x) & \frac{\partial}{\partial x} (-\sin 2y) \\ \frac{\partial}{\partial y} (\sin 2x) & \frac{\partial}{\partial y} (-\sin 2y) \end{bmatrix} = \begin{bmatrix} 2\cos 2x & 0 \\ 0 & -2\cos 2y \end{bmatrix}$$

$$\text{At } (x, y) = (0, \pi/2)$$

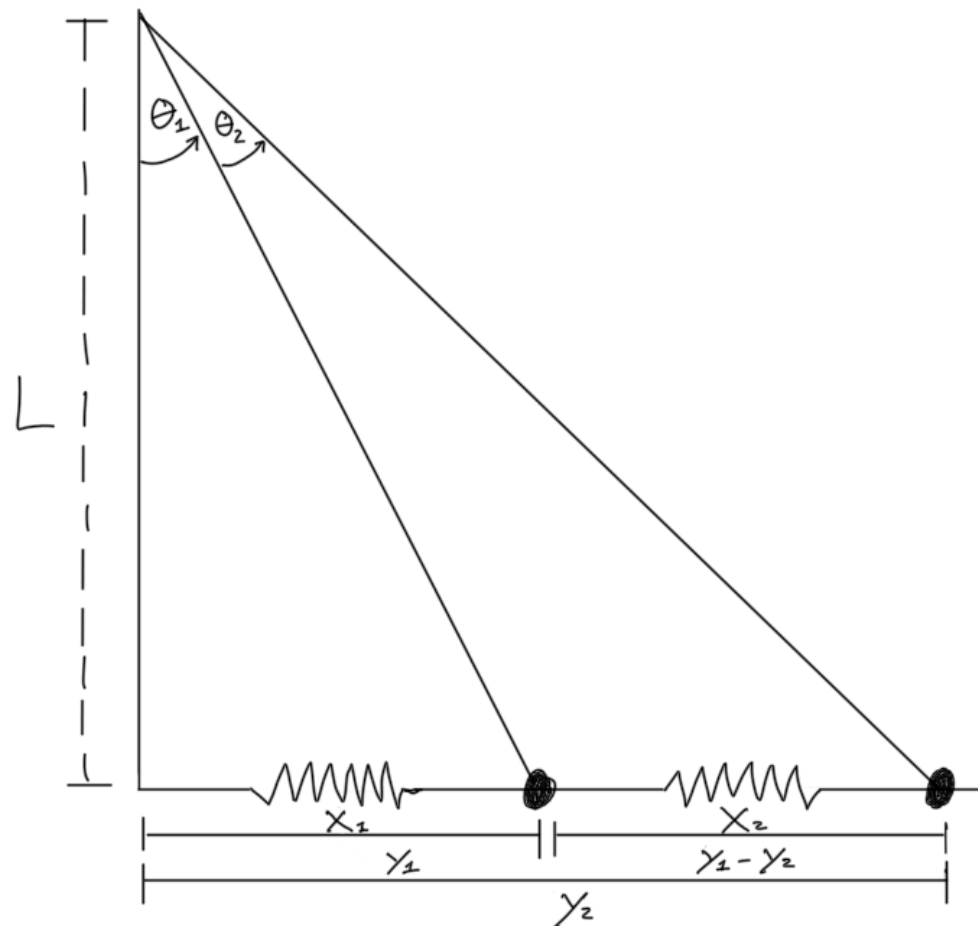
$$Df(x, y) = (\sin(0) \quad -\sin(\pi)) = (0 \quad 0)$$

$$D^2 f(x, y) = \begin{pmatrix} 2\cos(0) & 0 \\ 0 & -2\cos(\pi) \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$$

At minima, the necessary condition is $Df(x, y) = 0$ and sufficient condition is $D^2f(x, y) > 0$. For the given problem, at $(x, y) = (0, \frac{\pi}{2})$, $Df(x, y) = 0$ and $\text{Det}(D^2f(x, y)) = 4 > 0$

Problem 2 (20pts)

```
In [7]: #@title
from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master/twolinear masses.png' width=500' height='350'></td></tr></table>"))
```



Compute the equations of motion for the two-mass-spring system (shown above) in $\theta = (\theta_1, \theta_2)$ coordinates. The first sphere with mass m_1 is the one close to the wall, and the second sphere has mass m_2 . Assume that there is a spring of spring constant k_1 between the first mass and the wall and a spring of spring constant k_2 between the first mass and the second mass. The natural length of each spring is zero.

Turn in: Include the code used to symbolically solve for the equations of motion and the code output, which should be the equations of motion. Make sure you are using *SimPy*'s `.simplify()` functionality when printing your output.

```
In [8]: # Define the variables and parameters
t, g, m1, m2, k1, k2, L = sym.symbols(r't, g, m_1, m_2, k_1, k_2, L')

th1 = sym.Function(r'\theta_1')(t)
th2 = sym.Function(r'\theta_2')(t)

x1 = L*sym.tan(th1)
x2 = L*sym.tan(th1 + th2) - x1
```



```

x1_dot = x1.diff(t)
x2_dot = x2.diff(t)

PE = k1*(x1**2)/2 + k2*(x2**2)/2
KE = m1*(x1_dot**2)/2 + m2*((x2_dot + x1_dot)**2)/2

q = sym.Matrix([th1, th2])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Lagrangian
lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display equations
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

print('\033[1mEuler Lagrangian Equations')
EL = dLdq - dLdqdot.diff(t)

for exp, n in zip(EL, range(1,3)):
    print("\033[1mEquation {}".format(n))
    display(sym.simplify(sym.Eq(exp, 0)))

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0 for exp in EL])

# Define the equations
eqns = sym.Eq(lhs, rhs)

solns = sym.solve(eqns, qddot, dict=True)
# display(solutions)

for sol in solns:
    print('\n\033[1mSolution: ')
    for q in qddot:
        display(sym.simplify(sym.Eq(q, sol[q])))
    print("\n")

```

Lagrangian of the system:

$$L^2 \left(-k_1 \tan^2(\theta_1(t)) - k_2 (\tan(\theta_1(t) + \theta_2(t)) - \tan(\theta_1(t)))^2 + \frac{m_1 \left(\frac{d}{dt} \theta_1(t) \right)^2}{\cos^4(\theta_1(t))} + m_2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1)^2 \left(\frac{d}{dt} \theta_1(t) + \frac{d}{dt} \theta_2(t) \right)^2 \right)$$

Euler Lagrangian Equations

Equation 1

$$\frac{L^2 \left(-2m_1 \sin(\theta_1(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 - m_1 \cos(\theta_1(t)) \frac{d^2}{dt^2} \theta_1(t) - m_2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1)^2 \left(4 \left(\frac{d}{dt} \theta_1(t) + \frac{d}{dt} \theta_2(t) \right)^2 \tan(\theta_1(t) + \theta_2(t)) + \frac{d^2}{dt^2} \theta_1(t) + \frac{d^2}{dt^2} \theta_2(t) \right) \cos^5(\theta_1(t)) + \left(-\frac{k_1 \tan(\theta_1(t))}{\cos^2(\theta_1(t))} - k_2 (\tan(\theta_1(t) + \theta_2(t)) - \tan(\theta_1(t))) (\tan^2(\theta_1(t) + \theta_2(t)) - \tan^2(\theta_1(t))) + 2m_2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1)^2 \left(\frac{d}{dt} \theta_1(t) + \frac{d}{dt} \theta_2(t) \right)^2 \tan(\theta_1(t) + \theta_2(t)) \right) \cos^5(\theta_1(t)) \right)}{\cos^5(\theta_1(t))} = 0$$

Equation 2

$$L^2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1) \left(-k_2 (\tan(\theta_1(t) + \theta_2(t)) - \tan(\theta_1(t))) - 2m_2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1) \left(\frac{d}{dt} \theta_1(t) + \frac{d}{dt} \theta_2(t) \right)^2 \tan(\theta_1(t) + \theta_2(t)) - m_2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1) \left(\frac{d^2}{dt^2} \theta_1(t) + \frac{d^2}{dt^2} \theta_2(t) \right) \right) = 0$$

Solution:

$$\frac{d^2}{dt^2} \theta_1(t) = \frac{-2m_1 \sin(\theta_1(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 + (-k_1 \tan^3(\theta_1(t)) - k_1 \tan(\theta_1(t)) + k_2 \tan(\theta_1(t) + \theta_2(t)) \tan^2(\theta_1(t)) + k_2 \tan(\theta_1(t) + \theta_2(t)) - k_2 \tan^3(\theta_1(t)) - k_2 \tan(\theta_1(t))) \cos^5(\theta_1(t))}{m_1 \cos(\theta_1(t))}$$

$$\frac{d^2}{dt^2} \theta_2(t) = \frac{k_1 m_2 \cos^5(\theta_1(t)) \tan^2(\theta_1(t) + \theta_2(t)) \tan^3(\theta_1(t)) + k_1 m_2 \cos^5(\theta_1(t)) \tan^2(\theta_1(t) + \theta_2(t)) \tan(\theta_1(t)) + k_1 m_2 \cos^5(\theta_1(t)) \tan^3(\theta_1(t)) + k_1 m_2 \cos^5(\theta_1(t)) \tan(\theta_1(t)) + k_2 m_1 \sin(\theta_1(t)) - k_2 m_1 \cos(\theta_1(t)) \tan(\theta_1(t) + \theta_2(t)) - k_2 m_2 \cos^5(\theta_1(t)) \tan^3(\theta_1(t) + \theta_2(t)) \tan^2(\theta_1(t)) - k_2 m_2 \cos^5(\theta_1(t)) \tan^3(\theta_1(t) + \theta_2(t)) + k_2 m_2 \cos^5(\theta_1(t)) \tan^2(\theta_1(t) + \theta_2(t)) \tan^3(\theta_1(t)) + k_2 m_2 \cos^5(\theta_1(t)) \tan^2(\theta_1(t) + \theta_2(t)) \tan(\theta_1(t)) - k_2 m_2 \cos^5(\theta_1(t)) \tan(\theta_1(t) + \theta_2(t)) \tan^2(\theta_1(t)) - k_2 m_2 \cos^5(\theta_1(t)) \tan(\theta_1(t) + \theta_2(t)) + k_2 m_2 \cos^5(\theta_1(t)) \tan^3(\theta_1(t)) + k_2 m_2 \cos^5(\theta_1(t)) \tan(\theta_1(t)) + 2m_1 m_2 \sin(\theta_1(t)) \tan^2(\theta_1(t) + \theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 + 2m_1 m_2 \sin(\theta_1(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 - 2m_1 m_2 \cos(\theta_1(t)) \tan^3(\theta_1(t) + \theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 - 4m_1 m_2 \cos(\theta_1(t)) \tan^3(\theta_1(t) + \theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) - 2m_1 m_2 \cos(\theta_1(t)) \tan^3(\theta_1(t) + \theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2 - 2m_1 m_2 \cos(\theta_1(t)) \tan(\theta_1(t) + \theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 - 4m_1 m_2 \cos(\theta_1(t)) \tan(\theta_1(t) + \theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) - 2m_1 m_2 \cos(\theta_1(t)) \tan(\theta_1(t) + \theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2}{m_1 m_2 (\tan^2(\theta_1(t) + \theta_2(t)) + 1) \cos(\theta_1(t))}$$

Problem 3 (10pts)

For the same two-spring-mass system in Problem 2, show by example that Newton's equations do not hold in an arbitrary choice of coordinates (but they do, of course, hold in Cartesian coordinates). Your example should be implemented using Python's SymPy package.

Hint 1: In other words, you need to find a set of coordinates $q = [q_1, q_2]$, and compute the equations of motion ($F = ma = m\ddot{q}$), showing that these equations of motion do not make the same prediction as Newton's laws in the Cartesian inertially fixed frame (where they are correct).

Hint 2: Newton's equations don't hold in non-inertia coordinates. For the x_1, x_2 and y_1, y_2 coordinates shown in the image, one of them is non-inertia coordinate.

Turn in: Include the code you used to symbolically compute the equations of motion to show that Newton's equations don't hold. Also, include the output of the code, which should be the equations of motion under the chosen set of coordinates. Make sure to indicate what coordinate you choose in the comments.

```
In [9]: # Define the variables and parameters
t, g, m1, m2, k1, k2, L = sym.symbols('t, g, m_1, m_2, k_1, k_2, L')

x1 = sym.Function('x_1')(t)
x2 = sym.Function('x_2')(t)

y1 = sym.Function('y_1')(t)
y2 = sym.Function('y_2')(t)

x1_dot = x1.diff(t)
x2_dot = x2.diff(t)
```

```

y1_dot = y1.diff(t)
y2_dot = y2.diff(t)

x1_ddot = x1_dot.diff(t)
x2_ddot = x2_dot.diff(t)

y1_ddot = y1_dot.diff(t)
y2_ddot = y2_dot.diff(t)

# Writing Newton's Laws of motion in Inertial Coordinates (y1, y2)
print("\033[1mNewton equations in the inertial coordinates")
display(sym.simplify(sym.Eq(y1_ddot, (k2*y2/m1) - ((k1+k2)*y1/m1))))
display(sym.simplify(sym.Eq(y2_ddot, -((k2)*(y2-y1)/m2))))
print("\n")

# Non-Inertial Coordinates (x1, x2)
print("\033[1mNewton equations in the non-inertial coordinates")
display(sym.simplify(sym.Eq(x1_ddot, (k2*x2/m1) - (k1*x1/m1))))
display(sym.simplify(sym.Eq(x2_ddot, -(k2*x2/m2))))

```

Newton equations in the inertial coordinates

$$\frac{d^2}{dt^2}y_1(t) = \frac{k_2 y_2(t) - (k_1 + k_2) y_1(t)}{m_1}$$

$$\frac{d^2}{dt^2}y_2(t) = \frac{k_2 (y_1(t) - y_2(t))}{m_2}$$

Newton equations in the non-inertial coordinates

$$\frac{d^2}{dt^2}x_1(t) = \frac{-k_1 x_1(t) + k_2 x_2(t)}{m_1}$$

$$\frac{d^2}{dt^2}x_2(t) = -\frac{k_2 x_2(t)}{m_2}$$

Deriving the equations of motion by solving the E-L equations

```

In [10]: def solve_lagrangian(q, KE, PE):

    qdot = q.diff(t)
    qddot = qdot.diff(t)

    # Lagrangian
    lagrangian = KE - PE
    lagrangian = sym.simplify(lagrangian)

    # Display equations
    print('\033[1mLagrangian of the system: ')
    display(lagrangian)
    # print("\n")

    # Wrap the lagrangian into a SymPy Matrix
    L = sym.Matrix([lagrangian])
    # display(L)

    # Compute Jacobian
    dLdq = L.jacobian(q)
    dLdqdot = L.jacobian(qdot)

```

```

# print('\033[1mEuler Lagrangian Equations')
EL = dLdq - dLdqdot.diff(t)

# for exp, n in zip(EL, range(1,3)):
#     print("\033[1mEquation {}".format(n))
#     display(sym.simplify(sym.Eq(exp, 0)))

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0 for exp in EL])

# Define the equations
eqns = sym.Eq(lhs, rhs)

solns = sym.solve(eqns, qddot, dict=True)
# display(solutions)

for sol in solns:
    print('\n\033[1mSolution: ')
    for q in qddot:
        display(sym.simplify(sym.Eq(q, sol[q])))
        print("\n")
    return

# Define the variables and parameters
t, g, m1, m2, k1, k2, L = sym.symbols(r't, g, m_1, m_2, k_1, k_2, L')

x1 = sym.Function(r'x_1')(t)
x2 = sym.Function(r'x_2')(t)

y1 = sym.Function(r'y_1')(t)
y2 = sym.Function(r'y_2')(t)

x1_dot = x1.diff(t)
x2_dot = x2.diff(t)

y1_dot = y1.diff(t)
y2_dot = y2.diff(t)

# Non-Inertial Coordinates (x1, x2)
q = sym.Matrix([x1, x2])

PE = k1*(x1**2)/2 + k2*(x2**2)/2
KE = m1*(x1_dot**2)/2 + m2*((x2_dot + x1_dot)**2)/2

print("\033[1mE-L equations in the non-inertial coordinates")
solve_lagrangian(q, KE, PE)

# Inertial Coordinates (y1, y2)
q = sym.Matrix([y1, y2])

PE = k1*(y1**2)/2 + k2*((y2-y1)**2)/2
KE = m1*(y1_dot**2)/2 + m2*(y2_dot**2)/2

print("\033[1mE-L equations in the inertial coordinates")
solve_lagrangian(q, KE, PE)

```

E-L equations in the non-inertial coordinates
Lagrangian of the system:

$$-\frac{k_1x_1^2(t)}{2}-\frac{k_2x_2^2(t)}{2}+\frac{m_1\left(\frac{d}{dt}x_1(t)\right)^2}{2}+\frac{m_2\left(\frac{d}{dt}x_1(t)+\frac{d}{dt}x_2(t)\right)^2}{2}$$

Solution:

$$\frac{d^2}{dt^2}x_1(t)=\frac{-k_1x_1(t)+k_2x_2(t)}{m_1}$$

$$\frac{d^2}{dt^2}x_2(t)=\frac{k_1x_1(t)}{m_1}-\frac{k_2x_2(t)}{m_2}-\frac{k_2x_2(t)}{m_1}$$

E-L equations in the inertial coordinates
Lagrangian of the system:

$$-\frac{k_1y_1^2(t)}{2}-\frac{k_2(y_1(t)-y_2(t))^2}{2}+\frac{m_1\left(\frac{d}{dt}y_1(t)\right)^2}{2}+\frac{m_2\left(\frac{d}{dt}y_2(t)\right)^2}{2}$$

Solution:

$$\frac{d^2}{dt^2}y_1(t)=\frac{-k_1y_1(t)-k_2y_1(t)+k_2y_2(t)}{m_1}$$

$$\frac{d^2}{dt^2}y_2(t)=\frac{k_2\left(y_1(t)-y_2(t)\right)}{m_2}$$

Observation

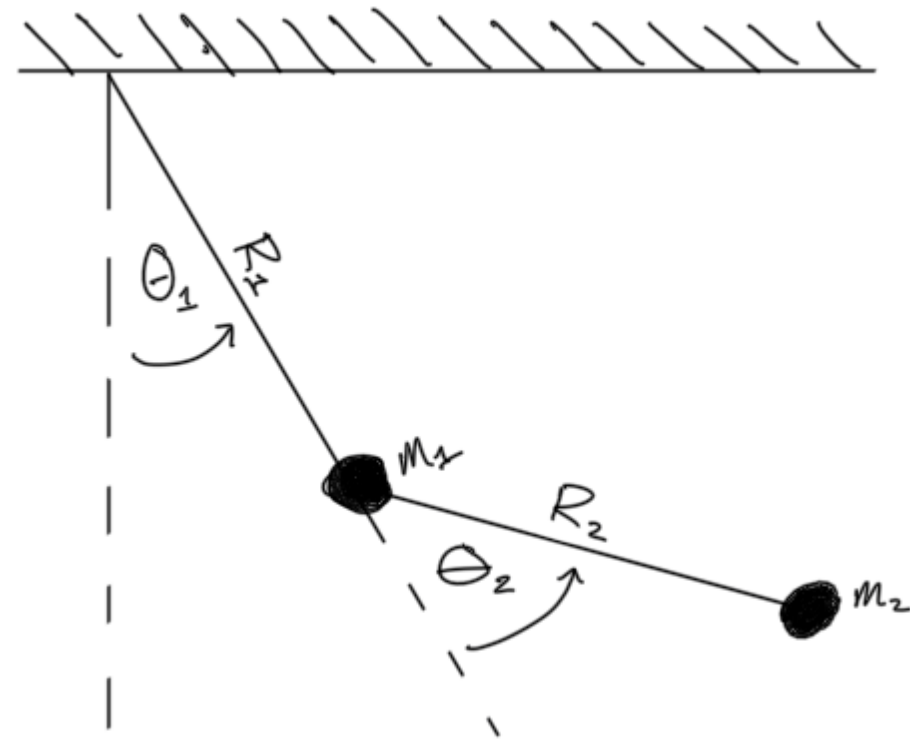
The equations of motion for the given two point mass spring system has been derived using two techniques in two different coordinate frames: Inertial and Non-Inertial

The equations derived using E-L method are equivalent when we substitute $y_2 = x_1 + x_2$ and $x_1 = y_1$ but the same is not applicable for the equations of motion derived using Newton's laws of motion specifiially for $\ddot{x}_2$.

This is because $\ddot{x}_2$ is being measured from a non-intertial frame $\ddot{x}_1$ i.e. it is accelerating w.r.t time. In this particular scenario, we can correct this by considering pseudo forces while writing the equations of motion for $\ddot{x}_2$

Problem 4 (10pts)

```
In [11]: #@title
from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master/dyndoublepend.png' width=500' height='350'></td></tr></table>"))
```



For the same double-pendulum system hanging in gravity in Homework 2 (shown above), take $q = [\theta_1, \theta_2]$ as the system configuration variables, with $R_1 = R_2 = 1, m_1 = m_2 = 1$. Symbolically compute the Hamiltonian of this system using Python's *SymPy* package.

Turn in: Include the code used to symbolically compute the Hamiltonian of the system and the code output, which should be the Hamiltonian of the system. Make sure you are using *SymPy*'s `.simplify()` functionality when printing your output.

```
In [12]: # Define the variables and parameters
t, g, m1, m2, R1, R2 = sym.symbols('t, g, m_1, m_2, R_1, R_2')

th1 = sym.Function(r'\theta_1')(t)
th2 = sym.Function(r'\theta_2')(t)

x1 = R1*sym.sin(th1)
y1 = -R1*sym.cos(th1)

x2 = R1*sym.sin(th1) + R2*sym.sin(th1+th2)
y2 = -R1*sym.cos(th1) - R2*sym.cos(th1+th2)

x1_dot = x1.diff(t)
y1_dot = y1.diff(t)

x2_dot = x2.diff(t)
y2_dot = y2.diff(t)

KE = (m1*(x1_dot**2 + y1_dot**2))/2 + (m2*(x2_dot**2 + y2_dot**2))/2
PE = m1*g*y1 + m2*g*y2

q = sym.Matrix([th1, th2])
qdot = q.diff(t)
qddot = qdot.diff(t)
```



```

## Display equations
## Hamiltonian

hamiltonian = KE + PE
hamiltonian = sym.simplify(hamiltonian)

print('\033[1mHamiltonian of the system: ')
hamiltonian = hamiltonian.subs({m1:1, m2:1, R1:1, R2:1, g:9.8})
display(hamiltonian)

# Lagrangian

lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# print('\033[1mLagrangian of the system: ')
# display(lagrangian)

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

# print('\033[1mEuler Lagrangian Equations')
EL = dLdq - dLdqdot.diff(t)

# for exp, n in zip(EL, range(1,3)):
#     print("\033[1mEquation {}".format(n))
#     display(sym.Eq(exp, 0))

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0 for exp in EL])

# Define the equations
eqns = sym.Eq(lhs, rhs)

solns = sym.solve(eqns, qddot, dict=True)
# display(solutions)

for sol in solns:
    print('\n\033[1mSolution: ')
    for q in qddot:
        display(sym.Eq(q, sol[q]))
    print("\n")

print('\033[1mSubstituting the constants m1 = 1, m2 = 1, R1 = 1 R2 = 1 and g = 9.8:')
c_sub_solns = [{q: soln[q].subs({m1:1, m2:1, R1:1, R2:1, g:9.8}) for soln in solns for q in qddot}]

for c_sub_soln in c_sub_solns:
    print('\n\033[1mSolution: ')
    for q in qddot:
        display(sym.simplify(sym.Eq(q, c_sub_soln[q])))
    print("\n")

```

```
# Lamdifying the constant substituted solutions for theta1_ddot and theta2_ddot
lambdified_solns = [sym.lambdify([th1, qdot[0], th2, qdot[1]], c_sub_soln[q]) for c_sub_soln in c_sub_solns for q in qddot]
th1_ddot, th2_ddot = lambdified_solns

lambdified_hamiltonian = sym.lambdify([th1, qdot[0], th2, qdot[1]], hamiltonian)
```

Hamiltonian of the system:

$$-9.8 \cos(\theta_1(t) + \theta_2(t)) - 19.6 \cos(\theta_1(t)) + \cos(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 + \cos(\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + \frac{3 \left(\frac{d}{dt} \theta_1(t) \right)^2}{2} + \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + \frac{\left(\frac{d}{dt} \theta_2(t) \right)^2}{2}$$

Solution:

$$\frac{d^2}{dt^2} \theta_1(t) = -\frac{R_1 m_2 \sin(\theta_2(t)) \cos(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2} - \frac{R_2 m_2 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2} - \frac{2 R_2 m_2 \sin(\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t)}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2} - \frac{R_2 m_2 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2} + \frac{g m_1 \sin(\theta_1(t))}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2} - \frac{g m_2 \sin(\theta_1(t) + \theta_2(t)) \cos(\theta_2(t))}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2} + \frac{g m_2 \sin(\theta_1(t))}{-R_1 m_1 + R_1 m_2 \cos^2(\theta_2(t)) - R_1 m_2}$$

$$\begin{aligned} \frac{d^2}{dt^2} \theta_2(t) = & \frac{R_1^2 m_1 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{R_1^2 m_2 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{2 R_1 R_2 m_2 \sin(\theta_2(t)) \cos(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{2 R_1 R_2 m_2 \sin(\theta_2(t)) \cos(\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t)}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} \\ & + \frac{R_1 R_2 m_2 \sin(\theta_2(t)) \cos(\theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{R_1 g m_1 \sin(\theta_1(t) + \theta_2(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} - \frac{R_1 g m_1 \sin(\theta_1(t)) \cos(\theta_2(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{R_1 g m_2 \sin(\theta_1(t) + \theta_2(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} \\ & - \frac{R_1 g m_2 \sin(\theta_1(t)) \cos(\theta_2(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{R_2^2 m_2 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{2 R_2^2 m_2 \sin(\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t)}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{R_2^2 m_2 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} \\ & - \frac{R_2 g m_1 \sin(\theta_1(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} + \frac{R_2 g m_2 \sin(\theta_1(t) + \theta_2(t)) \cos(\theta_2(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} - \frac{R_2 g m_2 \sin(\theta_1(t))}{-R_1 R_2 m_1 + R_1 R_2 m_2 \cos^2(\theta_2(t)) - R_1 R_2 m_2} \end{aligned}$$

Substituting the constants m1 = 1, m2 = 1, R1 = 1 R2 = 1 and g = 9.8:

Solution:

$$\begin{aligned} \frac{d^2}{dt^2} \theta_1(t) = & \frac{-4.9 \sin(\theta_1(t) + 2\theta_2(t)) + 14.7 \sin(\theta_1(t)) - 1.0 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 - 2.0 \sin(\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) - 1.0 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2 - 0.5 \sin(2\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2}{\cos^2(\theta_2(t)) - 2} \\ \\ & -9.8 \sin(\theta_1(t) - \theta_2(t)) + 9.8 \sin(\theta_1(t) + \theta_2(t)) + 4.9 \sin(\theta_1(t) + 2\theta_2(t)) - 14.7 \sin(\theta_1(t)) + 3.0 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 + 2.0 \sin(\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 1.0 \sin(\theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2 + 1.0 \sin(2\theta_2(t)) \left(\frac{d}{dt} \theta_1(t) \right)^2 + 1.0 \sin(2\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) \\ \frac{d^2}{dt^2} \theta_2(t) = & \frac{(2\theta_2(t)) \frac{d}{dt} \theta_1(t) \frac{d}{dt} \theta_2(t) + 0.5 \sin(2\theta_2(t)) \left(\frac{d}{dt} \theta_2(t) \right)^2}{\cos^2(\theta_2(t)) - 2} \end{aligned}$$

Problem 5 (10pts)

Simulate the double-pendulum system in Problem 4 with initial condition $\theta_1 = \theta_2 = -\frac{\pi}{2}, \dot{\theta}_1 = \dot{\theta}_2 = 0$ for $t \in [0, 10]$ and $dt = 0.01$. Numerically evaluate the Hamiltonian of this system from the simulated trajectory, and plot it.

Hint 1: The Hamiltonian can be numerically evaluated as a function of $\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2$, which means for each time step in the simulated trajectory, you can compute the Hamiltonian for this time step, and store it in a list or array for plotting later. This doesn't need to be done during the numerical simulation, after you have the simulated the trajectory you can access each time step within another loop.

Turn in: Include the code used to numerically evaluate and plot the Hamiltonian, as well as the code output, which should be the plot of Hamiltonian. Make sure you label the plot with axis labels, legend and a title.

```
In [13]: # Dynamics of the system
def double_pendulum(x):
    """
    System dynamics function (extended)

    Parameters
    =====
    x_dot: NumPy array
        s = [th1, th1_dot, th2, th2_ddot] is the extended system
        state vector

    Return
    =====
    x_ddot: NumPy array
        time derivative of input state vector,
        x_ddot = [th1_dot, th1_ddot, th2_dot, th2_ddot]
    """

    th1_k, th1_dot_k, th2_k, th2_dot_k = x

    th1_ddot_k = th1_ddot(th1_k, th1_dot_k, th2_k, th2_dot_k)
    th2_ddot_k = th2_ddot(th1_k, th1_dot_k, th2_k, th2_dot_k)

    return np.array([th1_dot_k, th1_ddot_k, th2_dot_k, th2_ddot_k])

def compute_hamiltonian_traj(x_traj):
    """
    System dynamics function (extended)

    Parameters
    =====
    x_dot: NumPy array
        s = [th1, th1_dot, th2, th2_ddot] is the extended system
        state vector

    Return
    =====
    x_ddot: NumPy array
        time derivative of input state vector,
        x_ddot = [th1_dot, th1_ddot, th2_dot, th2_ddot]
    """

    hamiltonian_traj = []
    for col in range(x_traj.shape[1]):
        th1_k, th1_dot_k, th2_k, th2_dot_k = x_traj[:, col]
        # print(x_traj[:, col])
        h_k = lambdified_hamiltonian(th1_k, th1_dot_k, th2_k, th2_dot_k)
        hamiltonian_traj.append(h_k)

    return np.array(hamiltonian_traj)

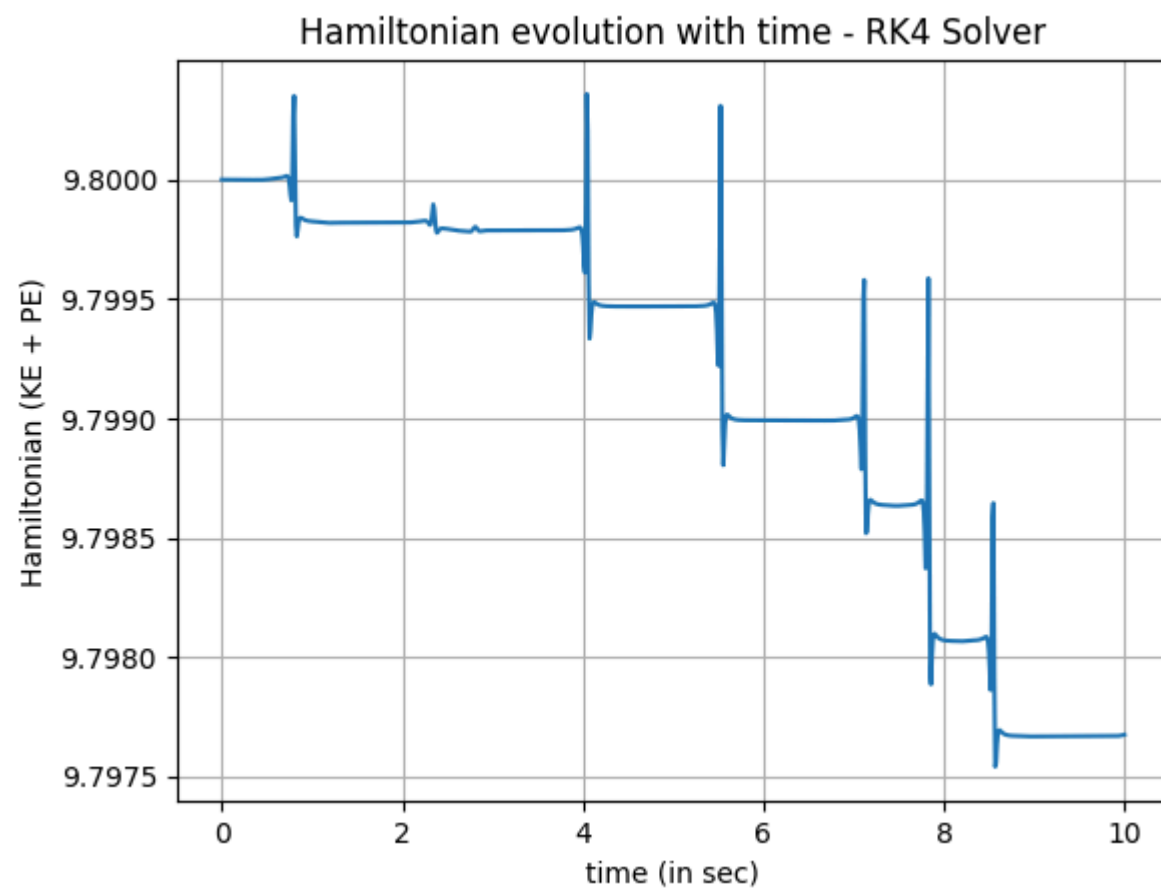
# State vector is of the form (th1, th1_dot, th2, th2_ddot)
# Define the initial state
x0 = np.array([-np.pi/2, 0, -np.pi/2, 0])
```

```
# Simulation parameters
t0 = 0
tf = 10
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

# print(double_pendulum(x0))
x_traj_rk4 = simulate(double_pendulum, x0, [t0, tf], dt, integrate)
# print(x_traj.shape)
hamiltonian_traj_rk4 = compute_hamiltonian_traj(x_traj_rk4)

# Set title and labels
plt.title(r"Hamiltonian evolution with time - RK4 Solver")
plt.xlabel("time (in sec)")
plt.ylabel("Hamiltonian (KE + PE)")
plt.grid(True)
plt.plot(tvec, hamiltonian_traj_rk4)
```

Out[13]: [



Problem 6 (15pts)

In the previously provided code for simulation, the numerical integration is a forth-order Runge–Kutta integration. Now, write down your own numerical integration function using Euler's method, and use your numerical integration function to simulate the same double-pendulum system with same parameters and initial condition in Problem 4. Compute and plot the Hamiltonian from the simulated trajectory, what's the difference between two plots?

Hint 1: You will need to implement a new `integrate()` function. This function takes in three inputs: a function $f(x)$ representing the dynamics of the system state x (you can consider it as $\dot{x} = f(x)$), current state x (for example $x(t)$ if t is the current time step), and integration step length dt . This function should output $x(t + dt)$, for which the analytical solution is $x(t + dt) = x(t) + \int_t^{t+dt} f(x(\tau))d\tau$. Thus, you need to think about how to numerically evaluate this integration using Euler's method.

Hint 2: The implemented function should have the same input-output structure as the previous one.

Hint 3: After you implement the new integration function, you can use the same helper function `simulate()` for simulation. You just need to input replace the integration function name as the new one (for example, your new function can be named as `euler_integrate()`). Please carefully read the comments in the `simulate()` function. Below is the template/example of how to implement the new integration function and use it for simulation.

Turn in: Include you numerical integration function (you only need to include the code for your new integration function), and the resulting plot of Hamiltonian. Make sure you label the plot appropriately with axis labels, legend and a title.

```
In [14]: #####
# Below is an example of how to implement a new integration
# function and use it for simulation

#####
# This is the same "simulate()" function we provided
# in previous homework.
def simulate(f, x0, tspan, dt, integrate):
    """
    This function takes in an initial condition x0, a timestep dt,
    a time span tspan consisting of a list [min_time, max_time],
    as well as a dynamical system f(x) that outputs a vector of the
    same dimension as x0. It outputs a full trajectory simulated
    over the time span of dimensions (xvec_size, time_vec_size).

    Parameters
    =====
    f: Python function
        derivate of the system at a given step x(t),
        it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
    x0: NumPy array
        initial conditions
    tspan: Python list
        tspan = [min_time, max_time], it defines the start and end
        time of simulation
    dt:
        time step for numerical integration
    integrate: Python function
        numerical integration method used in this simulation

    Return
    =====
    x_traj:
        simulated trajectory of x(t) from t=0 to tf
    """
    N = int((max(tspan)-min(tspan))/dt)
    x = np.copy(x0)
    tvec = np.linspace(min(tspan),max(tspan),N)
    xtraj = np.zeros((len(x0),N))
    for i in range(N):
        xtraj[:,i]=integrate(f,x,dt)
        x = np.copy(xtraj[:,i])
    return xtraj

#####
# This is the same "integrate()" function we provided in previous homework.
def integrate(f, xt, dt):
```

```

"""
This function takes in an initial condition x(t) and a timestep dt,
as well as a dynamical system f(x) that outputs a vector of the
same dimension as x(t). It outputs a vector x(t+dt) at the future
time step.

Parameters
=====
dyn: Python function
    derivate of the system at a given step x(t),
    it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
xt: NumPy array
    current step x(t)
dt:
    step size for integration

Return
=====
new_xt:
    value of x(t+dt) integrated from x(t)
"""
k1 = dt * f(xt)
k2 = dt * f(xt+k1/2.)
k3 = dt * f(xt+k2/2.)
k4 = dt * f(xt+k3)
new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
return new_xt

#####
# This is where you implement your new integration function for this
# problem. Please make sure the new integration function has the same
# input-output structure as the old one.
def euler_integrate(f, xt, dt):
    """
    This function takes in an initial condition x(t) and a timestep dt,
    as well as a dynamical system f(x) that outputs a vector of the
    same dimension as x(t). It outputs a vector x(t+dt) at the future
    time step.

    Parameters
    =====
    dyn: Python function
        derivate of the system at a given step x(t),
        it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
    xt: NumPy array
        current step x(t)
    dt:
        step size for integration

    Return
    =====
    new_xt:
        value of x(t+dt) integrated from x(t)
    """
    dx = dt * f(xt)
    new_xt = xt + dx
    return new_xt

#####
# In this example, we're going to simulate a particle falling in gravity,

```

```

# and assume that we already have the equations of motion (which you have
# to use Euler-Lagrange equations to solve for in the homework, but you
# should have that in last homework)
import numpy as np

def xddot(x, xdot):
    return -9.8

def dyn(s):
    return np.array([s[1], xddot(s[0], s[1])])

# define initial condition
s0 = np.array([10, 0])

#####
# We first use the old integration function to simulate the system, please
# carefully read the comments inside the "simulate()" function
traj_rk4 = simulate(f=dyn, x0=s0, tspan=[0,10], dt=0.01, integrate=integrate)
# note that, here I pass the function arguments explicitly using the so-called
# *keyword arguments*. It's not necessary, as long as the order of the arguments
# are correct, you don't have to explicitly write down the keyword names. When
# calling the function explicitly through keyword arguments, we have "integrate=integrate",
# the first "integrate" indicates the argument name (you can see it in function definition),
# and the second "integrate" is the name of the provided integration function.
# You can also call the function as: "traj = simulate(dyn, s0, [0,10], 0.01, integrate)",
# where "integrate" indicates the provided integration function name.

#####
# Based on the code above, once you finished your own integration function,
# which is named here as "euler_integrate" (but you really can name it
# as anything you like), you can simulate the trajectory as:
traj_euler = simulate(f=dyn, x0=s0, tspan=[0,10], dt=0.01, integrate=euler_integrate)
# if you run this code now, of course it will not succeed, since you haven't implemented
# "euler_integrate" yet.

```

```

In [15]: x_traj_euler = simulate(double_pendulum, x0, [t0, tf], dt, euler_integrate)
# print(x_traj.shape)
hamiltonian_traj_euler = compute_hamiltonian_traj(x_traj_euler)

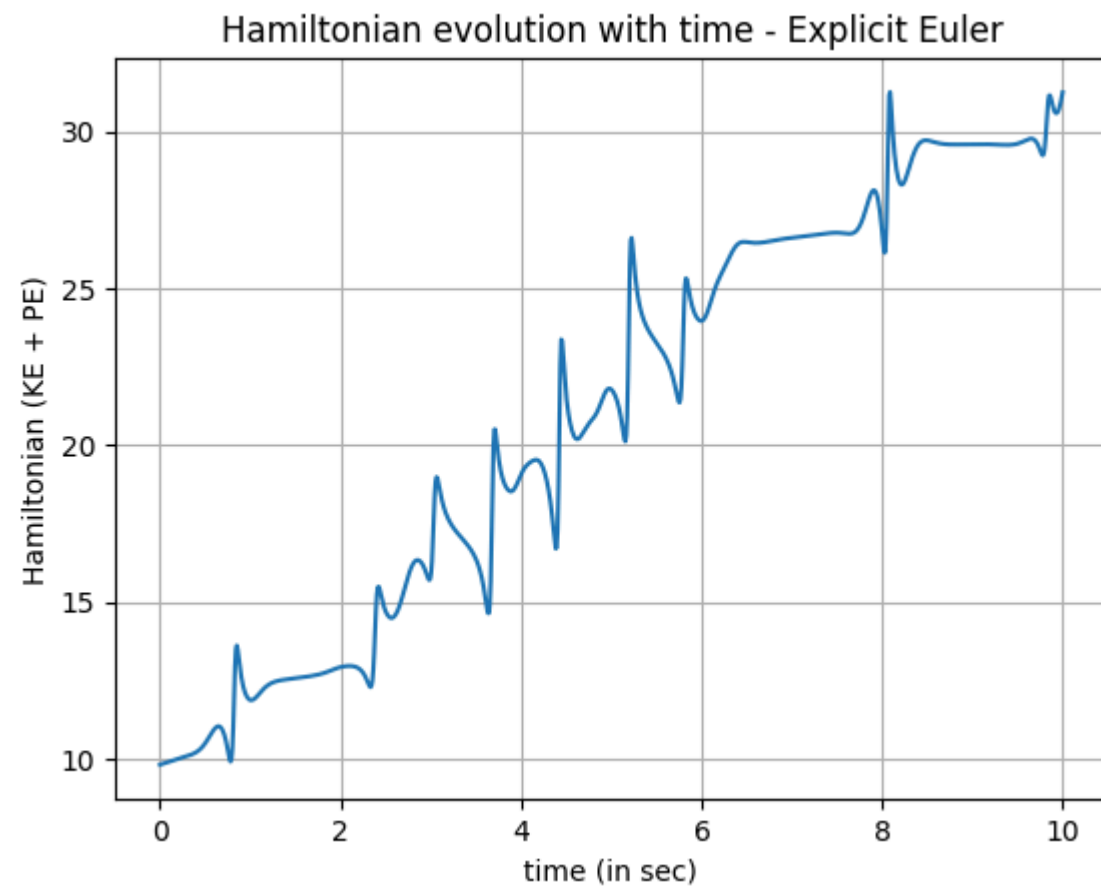
plt.title(r"Hamiltonian evolution with time - Explicit Euler")
plt.xlabel("time (in sec)")
plt.ylabel("Hamiltonian (KE + PE)")
plt.grid(True)
plt.plot(tvec, hamiltonian_traj_euler)

```

```

Out[15]: [<matplotlib.lines.Line2D at 0x7dd710ec4040>]

```



Observation

In the above example, the total energy is the Hamiltonian which is a conserved quantity and should remain constant over time.

- In the first plot, where the Hamiltonian is calculated using trajectories from the RK4 solver, the Hamiltonian shows an oscillatory behavior, and the average energy is decreasing with time, indicating that the solver exhibits dissipative behavior.
- In the second plot, where the Hamiltonian is calculated using trajectories from Explicit Euler integration, the Hamiltonian increasing with time, causing the system's average energy to rise, indicating instability in the solver.

Problem 7 (20pts)

For the same double-pendulum you simulated in Problem 4 with same parameters and initial condition, now add a constraint to the system such that the distance between the second pendulum and the origin is fixed at $\sqrt{2}$. Simulate the system with same parameters and initial condition, and animate the system with the same animate function provided in Homework 2.

Hint 1: What do you think the equations of motion should look like? Think about how the system will behave after adding the constraint. With no double, you can solve this problem using ϕ and all the following results for constrained Euler-Lagrange equations, however, if you really understand this constrained system, things might be much easier, and you can actually treat it as an unconstrained system.

Turn in: Include the code used to numerically evaluate, simulate and animate the system. Also, upload the video of animation separately through Canvas in ".mp4" format. You can use screen capture or record the screen directly with your phone.

```
In [16]: # Re-defining the variables
q = sym.Matrix([th1, th2])
qdot = q.diff(t)
qddot = qdot.diff(t)
```



```

# Define
lambda_ = sym.symbols(r'\lambda')

# Constraint
phi = x2**2 + y2**2 - 2
P = sym.Matrix([phi])
# display(P)
# display(dLdq)

phi_dot = phi.diff(t)
phi_ddot = phi_dot.diff(t)

dPdq = P.jacobian(q)

# print('\n\033[1mConstrained Euler Lagrangian Equations')
EL_constrained = dLdq - dLdqdot.diff(t)

lhs = []
rhs = []
qnew = []

# Create the lhs and rhs of the constrained EL equations
for l, r in zip(EL_constrained, dPdq):
    lhs.append(l)
    rhs.append(lambda_*r)

# Add the equation ddot(phi) = 0
lhs.append(phi_ddot)
rhs.append(0)

lhs = sym.Matrix(lhs)
rhs = sym.Matrix(rhs)

# Define the equations
eqns = sym.Eq(lhs, rhs)
# display(eqns)
# print("\n")

for q in qddot:
    qnew.append(q)
qnew.append(lambda_)

qnew = sym.Matrix(qnew)

solns = sym.solve(eqns, qnew, dict=True)
# display(sol)

# for soln in solns:
#     print('\n\033[1mSolution: ')
#     for q in qnew:
#         display(sym.Eq(q, soln[q]))
# print("\n")

print('\033[1mSubstituting the constants m1 = 1, m2 = 1, R1 = 1 R2 = 1 and g = 9.8:')
c_sub_solns = [{q: soln[q].subs({m1:1, m2:1, R1:1, R2:1, g:9.8}) for soln in solns for q in qnew}]

for c_sub_soln in c_sub_solns:
    print('\n\033[1mSolution: ')
    for q in qnew:
        display(sym.simplify(sym.Eq(q, c_sub_soln[q])))

```

```
print("\n")
```

```
# Lamdifying the constant substituted solutions for theta1_ddot and theta2_ddot
lambdified_solns = [sym.lambdify([th1, qdot[0], th2, qdot[1]], c_sub_soln[q]) for c_sub_soln in c_sub_solns for q in qnew]
th1_ddot_constrained, th2_ddot_constrained, lambda_constrained = lambdified_solns
```

Substituting the constants m1 = 1, m2 = 1, R1 = 1 R2 = 1 and g = 9.8:

Solution:

$$\frac{d^2}{dt^2}\theta_1(t) = \frac{-9.8 \cos(\theta_1(t) - \theta_2(t)) + 9.8 \cos(\theta_1(t) + \theta_2(t)) + 4.9 \cos(\theta_1(t) + 2\theta_2(t)) - 4.9 \cos(\theta_1(t)) + 1.0 \cos(\theta_2(t)) \left(\frac{d}{dt}\theta_2(t)\right)^2 - 1.0 \cos(2\theta_2(t)) \frac{d}{dt}\theta_1(t) \frac{d}{dt}\theta_2(t) + 1.0 \frac{d}{dt}\theta_1(t) \frac{d}{dt}\theta_2(t) + 1.0 \left(\frac{d}{dt}\theta_2(t)\right)^2}{3 \sin(\theta_2(t)) + \sin(2\theta_2(t))}$$

$$\frac{d^2}{dt^2}\theta_2(t) = -\frac{\left(\frac{d}{dt}\theta_2(t)\right)^2}{\tan(\theta_2(t))}$$

$$\lambda = \frac{-39.2 \cos(\theta_1(t) - 2\theta_2(t)) - 58.8 \cos(\theta_1(t) - \theta_2(t)) + 78.4 \cos(\theta_1(t) + \theta_2(t)) - 39.2 \cos(\theta_1(t) + 2\theta_2(t)) - 19.6 \cos(\theta_1(t) + 3\theta_2(t)) + 78.4 \cos(\theta_1(t)) + 4.0 \cos(\theta_2(t)) \left(\frac{d}{dt}\theta_1(t)\right)^2 + 4.0 \cos(\theta_2(t)) \frac{d}{dt}\theta_1(t) \frac{d}{dt}\theta_2(t) - 8.0 \cos(\theta_2(t)) \left(\frac{d}{dt}\theta_2(t)\right)^2 - 12.0 \cos(2\theta_2(t)) \left(\frac{d}{dt}\theta_1(t)\right)^2 - 8.0 \cos(2\theta_2(t)) \frac{d}{dt}\theta_1(t) \frac{d}{dt}\theta_2(t) - 4.0 \cos(2\theta_2(t)) \left(\frac{d}{dt}\theta_2(t)\right)^2 - 4.0 \cos(3\theta_2(t)) \left(\frac{d}{dt}\theta_1(t)\right)^2 - 4.0 \cos(3\theta_2(t)) \frac{d}{dt}\theta_1(t) \frac{d}{dt}\theta_2(t) + 12.0 \left(\frac{d}{dt}\theta_1(t)\right)^2 + 8.0 \frac{d}{dt}\theta_1(t) \frac{d}{dt}\theta_2(t) + 4.0 \left(\frac{d}{dt}\theta_2(t)\right)^2}{4(1 - \cos(2\theta_1(t)))^2 \cos(\theta_2(t)) + 12(1 - \cos(2\theta_1(t)))^2 \cos(2\theta_2(t)) - 4(1 - \cos(2\theta_1(t)))^2 \cos(3\theta_2(t)) + 12(1 - \cos(2\theta_1(t)))^2 - 4 \cos(2\theta_1(t) - 3\theta_2(t)) + 12 \cos(2\theta_1(t) - 2\theta_2(t)) + 4 \cos(2\theta_1(t) - \theta_2(t)) + 4 \cos(2\theta_1(t) + \theta_2(t)) + 12 \cos(2\theta_1(t) + 2\theta_2(t)) - 4 \cos(2\theta_1(t) + 3\theta_2(t)) + \cos(4\theta_1(t) - 3\theta_2(t)) - 3 \cos(4\theta_1(t) - 2\theta_2(t)) - \cos(4\theta_1(t) - \theta_2(t)) - \cos(4\theta_1(t) + \theta_2(t)) - 3 \cos(4\theta_1(t) + 2\theta_2(t)) + \cos(4\theta_1(t) + 3\theta_2(t)) + 24 \cos(2\theta_1(t)) - 6 \cos(4\theta_1(t)) + 2 \cos(\theta_2(t)) - 42 \cos(2\theta_2(t)) - 2 \cos(3\theta_2(t)) + 6}$$

```
In [17]: # Dynamics of the system
def constrained_double_pendulum(x):
    """
    System dynamics function (extended)

    Parameters
    =====
    x_dot: NumPy array
            s = [th1, th1_dot, th2, th2_ddot] is the extended system
            state vector

    Return
    =====
    x_ddot: NumPy array
            time derivative of input state vector,
            x_ddot = [th1_dot, th1_ddot, th2_dot, th2_ddot]
    """

    th1_k, th1_dot_k, th2_k, th2_dot_k = x

    th1_ddot_k = th1_ddot_constrained(th1_k, th1_dot_k, th2_k, th2_dot_k)
    th2_ddot_k = th2_ddot_constrained(th1_k, th1_dot_k, th2_k, th2_dot_k)

    return np.array([th1_dot_k, th1_ddot_k, th2_dot_k, th2_ddot_k])

# State vector is of the form (th1, th1_dot, th2, th2_ddot)
# Define the initial state
x0 = np.array([-np.pi/2, 0, -np.pi/2, 0])
```



```
# Simulation parameters
t0 = 0
tf = 10
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

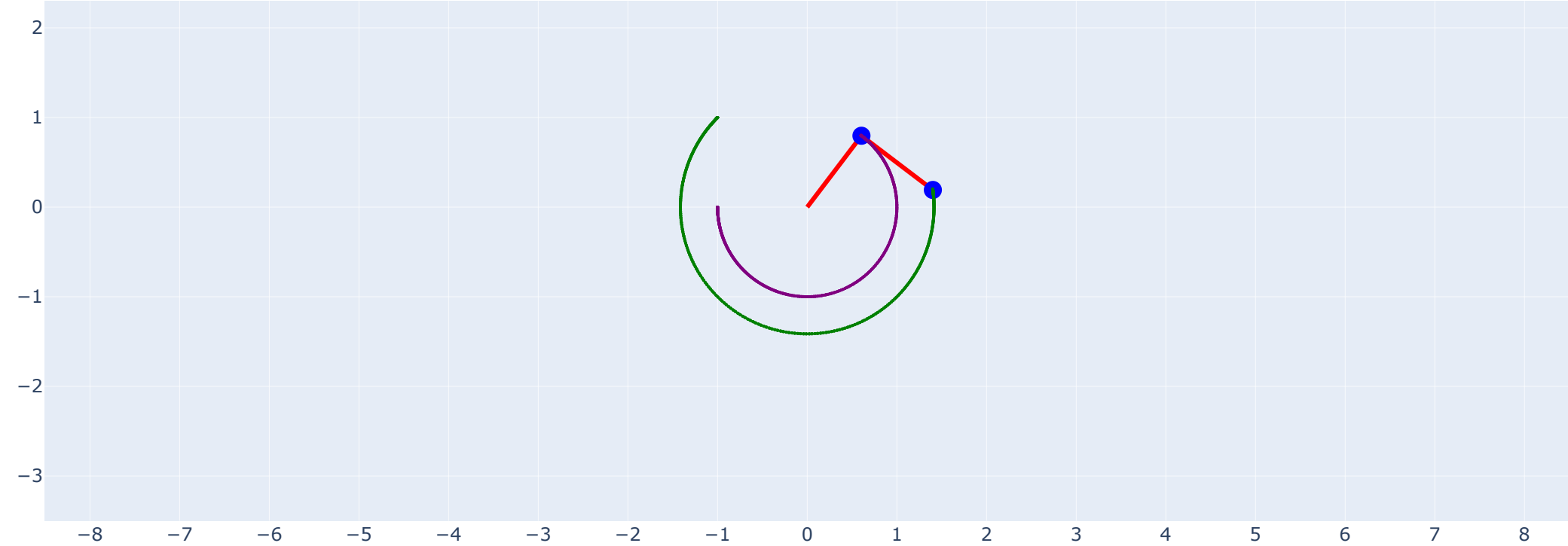
# print(double_pendulum(x0))
x_traj_constrained = simulate(constrained_double_pendulum, x0, [t0, tf], dt, integrate)
```

In [18]: `animate_double_pend(np.delete(x_traj_constrained, [1, 3], axis=0), L1=1, L2=1, T=10)`

Double Pendulum Simulation

Play

Pause



Problem 8 (5pts)

For the same system with same constraint in Problem 6, simulate the system with initial condition $\theta_1 = \theta_2 = -\frac{\pi}{4}$, which actually violates the constraint! Simulate the system and see what happen, what do you think is the actual influence after adding this constraint?

Turn in: Your thoughts about the actual effect of the constraint in this system. Note that you don't need to include any code for this problem.

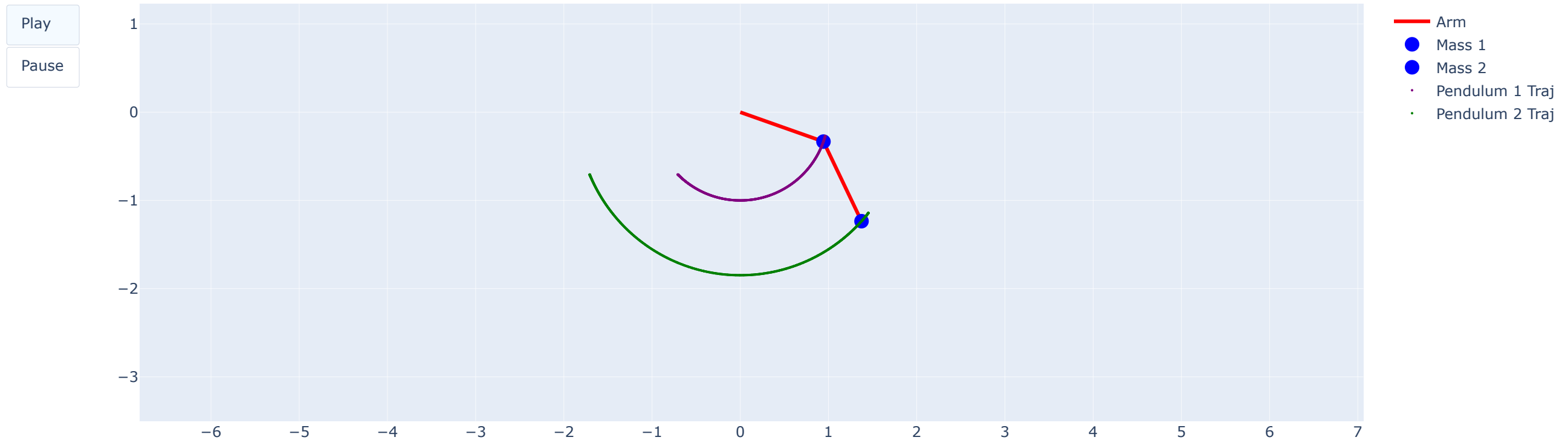
In [19]: `# State vector is of the form (th1, th1_dot, th2, th2_ddot)`
`# Define the initial state`
`x0 = np.array([-np.pi/4, 0, -np.pi/4, 0])`

```
# Simulation parameters
t0 = 0
tf = 10
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

# print(double_pendulum(x0))
x_traj_constrained_violated = simulate(constrained_double_pendulum, x0, [t0, tf], dt, integrate)
```

```
In [20]: animate_double_pend(np.delete(x_traj_constrained_violated, [1, 3], axis=0), L1=1, L2=1, T=10)
```

Double Pendulum Simulation



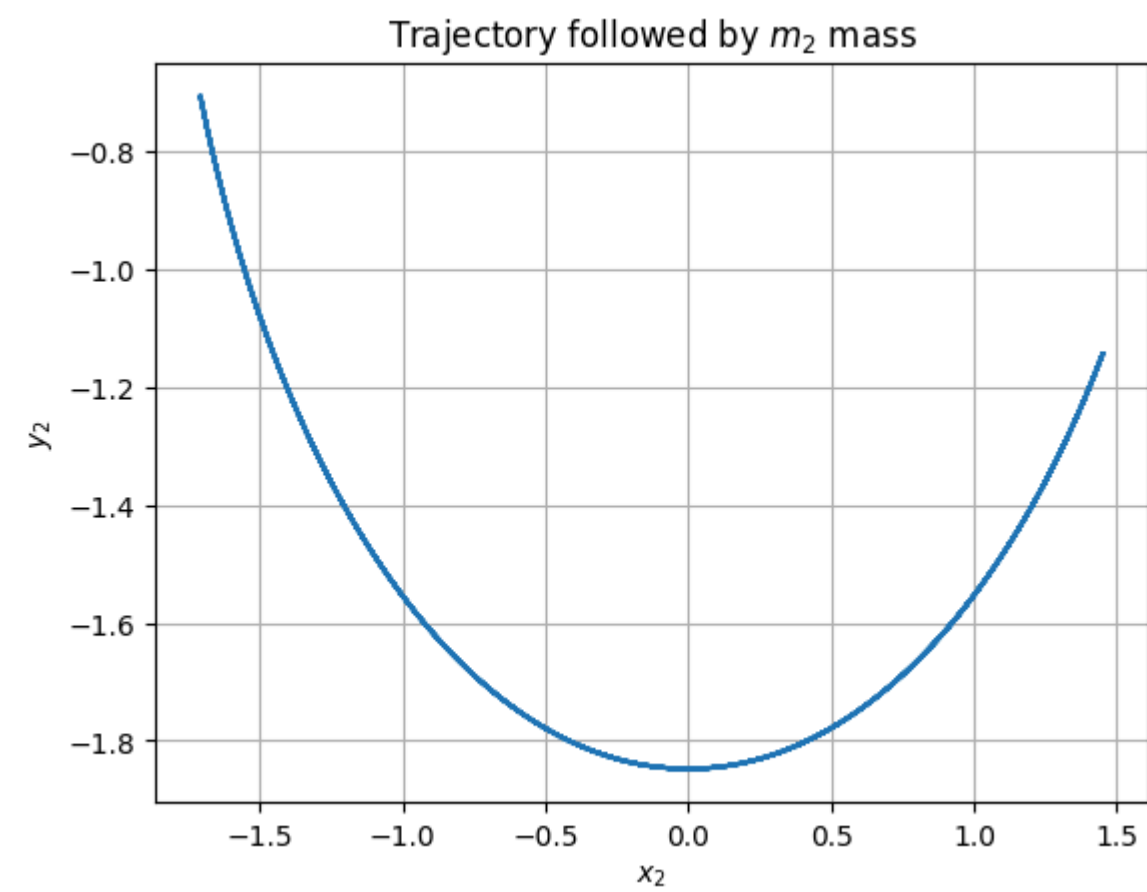
```
In [38]: x2 = lambda th1, th2: np.sin(th1) + np.sin(th1+th2)
y2 = lambda th1, th2: - np.cos(th1) - np.cos(th1+th2)

x2_traj = x2(x_traj_constrained_violated[0,:], x_traj_constrained_violated[2,:])
y2_traj = y2(x_traj_constrained_violated[0,:], x_traj_constrained_violated[2,:])

plt.xlabel("$x_2$")
plt.ylabel("$y_2$")
plt.grid(True)
plt.plot(x2_traj, y2_traj)
plt.title("Trajectory followed by $m_2$ mass")
# plt.plot(tvec, x_traj_constrained_violated[2,:], label=r"$\theta_2$")
# plt.plot(tvec, x_traj_constrained_violated[0,:], label=r"$\theta_1$")

# plt.legend()
```

```
Out[38]: Text(0.5, 1.0, 'Trajectory followed by $m_2$ mass')
```



Observation

The second mass continues to follow a curve due to the applied constraint force, but as shown in the plot, the curve is no longer a circle, and logically, it is no longer concentric with the trajectory of mass m_1

At any time t , there is sufficient constraint force acting on mass m_2 to keep it near the circle of radius $\sqrt{2}$. Although the initial condition is violated, the force still ensures that while the mass deviates from the circular path, it follows a controlled curve, preventing erratic behavior.